THE WORLD OF ALGORITHMS

Origin of the Word "Algorithm"

- Al-Khwārizmī (780–850 CE), a Persian mathematician at the "House of Wisdom" in Baghdad, admired Indian mathematics and learnt Sanskrit.
- Around 820 CE he wrote a detailed work explaining the Indian place-value system and the Indian methods for addition, subtraction, multiplication and division.
- The original is lost, but its 12th-century Latin translation, *Liber Algorismi de numero Indorum* (Al-Khwārizmī on Indian Numerals), survives. It spread Indian arithmetic through medieval (of the Middle Ages, about 500 to 1500 CE) Europe.
- From *Algoritmi*, the Latin form of his name, came the word *algorismus*, meaning any Indian method of calculation. It later became *algorism* and then **algorithm**.
- So "algorithm" first meant an Indian method of arithmetic; today it means any clear, step-by-step, systematic procedure for solving a class of problems.

Adding Numbers

✓ **Algorithm:**

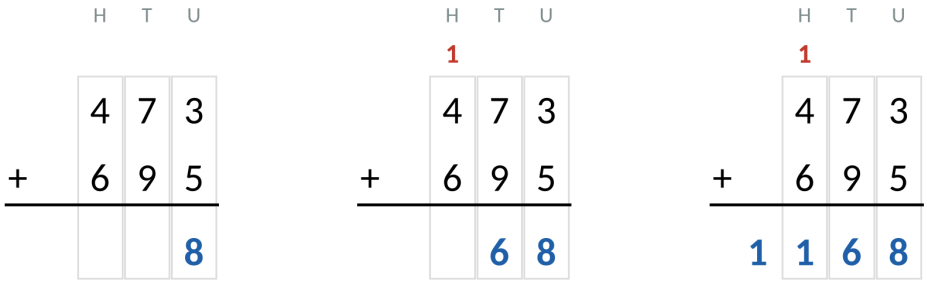
- **Algorithm:** a step-by-step procedure that leads to the answer of a problem.
- The steps are written as precisely (exactly, without any doubt) as possible, so that anyone who follows them exactly gets the correct answer.
- **Basic step:** a simple action that the algorithm assumes we can already do directly (for addition: adding two single digits).

✓ **Adding by counting:**

- To add 5 and 7 by counting: draw 5 dots, draw 7 more dots, then count all the dots: 12.
- This works, but for big numbers like 473 + 695 it means drawing and counting 1168 dots. It is very slow.

✓ **Adding digit by digit:**

- Write the numbers one below the other in the Indian (base-ten) place-value system, with the rightmost digits in one column.
- Add each column from right to left. If a column total is 10 or more, write its units digit and **carry 1** to the next column on the left.



Step 1: units
3 + 5 = 8, carry 0

Step 2: tens
7 + 9 + 0 = 16, write 6, carry 1

Step 3: hundreds
4 + 6 + 1 = 11, write 11

Fig. 1: Adding 473 + 695 column by column; the carry is shown in red

Conceptual Learning

✓ **Algorithm to add two numbers:**

1) Write the numbers one below the other in the base-ten place-value system, digits aligned from right to left.
2) Add the rightmost digits.
   - o If the sum is less than 10, write the sum below the two digits. Set carry = 0.
   - o If the sum is 10 or more, write the units digit of the sum below the two digits. Set carry = 1.
3) Move to the next column on the left. Add the two digits and the current carry, and use the same two rules as in Step 2.
4) Repeat Step 3 until there are no more digits on the left.
5) If carry = 1 at the end, write 1 to the left of the bottom row.

✓ **Why digit-by-digit addition is fast:**

- Going from 3-digit to 4-digit numbers gives about **10 times** as many dots to count, but only **1 more column** to add.
- Going from 3-digit to 5-digit numbers gives about **100 times** as many dots, but only **2 more columns**.
- So the work of the column method grows with the **number of digits**, while the work of counting grows with the **value** of the numbers.

✓ **Worked examples:**

- **Example 1 (time taken):** suppose we count 1 dot per second, and add one column (with carry) in 15 seconds.

Sum	Dots to count	Time by counting	Columns	Time by columns
33 + 27	60	1 minute	2	30 seconds
334 + 272	606	about 10 minutes	3	45 seconds
3347 + 2729	6076	over 1 hour 40 minutes	4	1 minute

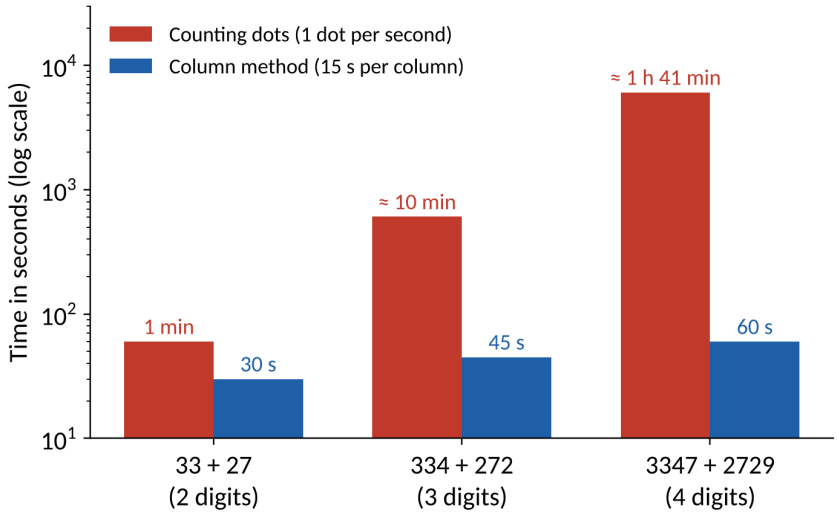


Fig. 2: Time taken: counting dots versus adding column by column

- **Example 2:** 2758 + 6467.
  - o Units: 8 + 7 = 15, write 5, carry 1.
  - o Tens: 5 + 6 + 1 = 12, write 2, carry 1.
  - o Hundreds: 7 + 4 + 1 = 12, write 2, carry 1.

Conceptual Learning

- Thousands: 2 + 6 + 1 = 9, write 9, carry 0. Answer: **9225**.

✓ **Why the addition algorithm works:**

- Each number is split into groups: units, tens, hundreds, and so on. For 473: 4 hundreds, 7 tens, 3 units.
- Adding column by column adds units to units, tens to tens and hundreds to hundreds.
- When a group reaches ten or more, ten of them make one group of the next size (10 units = 1 ten, 10 tens = 1 hundred). This new group is the **carry**.

	Hundreds	Tens	Units
473	4	7	3
695	6	9	5
Column totals	10	16	8
After carrying	10 + 1 = 11	16 tens = 1 hundred + 6 tens → 6	8

- So 473 + 695 = 11 hundreds + 6 tens + 8 units = **1168**.

✓ **Solved questions:**

Q1. Add two 4-digit numbers using the steps we have written down. Make sure you follow the steps precisely; do not perform any action that is not explicitly mentioned. Are you able to obtain the correct result?

- Take 3462 + 1548.
- Step 1: write 3462 above 1548, digits aligned.
- Step 2: units 2 + 8 = 10.
- The rule as first written says “less than 10” or “more than 10”; a sum of exactly 10 is not covered.
- So the steps must say “**10 or more**” (as in the corrected algorithm above)
- Then write 0, carry 1.
- Step 3: tens 6 + 4 + 1 = 11, write 1, carry 1.
- Hundreds 4 + 5 + 1 = 10, write 0, carry 1.
- Thousands 3 + 1 + 1 = 5, write 5, carry 0.
- Step 5: carry is 0, so nothing more is written.
- Result: **5010**, which is correct.
- So the steps give the correct result once the case “sum = 10” is included.

Q2. What happens if you add a 5-digit number to a 3-digit number? Do our steps handle this situation correctly?

- Example: 48273 + 659.
- After the hundreds column, the 3-digit number has no digits left, but Step 3 says “add the two digits”.
- The steps do not say what to do when only one digit is left in a column.
- Fix: treat a missing digit as **0** (write 00659)
- Then the columns give 8 + 0 + 1 = 9 and 4 + 0 = 4.
- Answer: 48273 + 659 = **48932**. With this fix, the steps work.

Q3. Why is it important to align the columns from right to left?

Conceptual Learning

- The rightmost digit of every number is the units digit, whatever the length of the number.
- Aligning from the right puts units under units, tens under tens, and so on.
- Only digits of the same place value can be added together.
- Aligning from the left would add, say, hundreds to tens and give a wrong answer.

Q4. In Step 3, why cannot the value of carry be more than 1?

- The biggest digits are 9 and 9, and the biggest carry coming in is 1.
- So the largest possible column sum is 9 + 9 + 1 = 19.
- 19 is less than 20, so a column can make at most one ten.
- The carry is always **0 or 1**.

Q5. What happens if we do not include the fifth step in the algorithm above? Give examples where the algorithm will work correctly and where it will fail to work.

- Without Step 5, a carry from the leftmost column is lost.
- **Works:** 123 + 456 = 579; the last column (1 + 4 = 5) gives no carry.
- **Fails:** 473 + 695: the hundreds give 4 + 6 + 1 = 11; we write 1 and carry 1, but the carry is never written.
- The answer comes out as 168 instead of **1168**.
- So Step 5 is needed whenever the leftmost column makes a carry.

Q6. See if you can complete the argument about grouping by units, tens, hundreds, ... to justify why the addition algorithm works.

- Any number can be written as a sum of groups: 473 = 4 × 100 + 7 × 10 + 3 × 1, and 695 = 6 × 100 + 9 × 10 + 5 × 1.
- Addition can be done in any order, so 473 + 695 = (4 + 6) × 100 + (7 + 9) × 10 + (3 + 5) × 1.
- This is exactly column-by-column addition: units with units, tens with tens, hundreds with hundreds.
- A column total of 10 or more contains a full group of ten, e.g. 16 tens = 1 × 100 + 6 × 10.
- Moving that one group to the next column is the carry.
- So the digits written are the correct units, tens, hundreds, ... of the total: 11 × 100 + 6 × 10 + 8 = **1168**.

Q7. How would you modify the algorithm to add two decimal fractions?

- Write the numbers one below the other with the **decimal points in one column** (not the rightmost digits).
- If one number has fewer decimal places, fill the empty places with zeros.
- Example: 12.5 + 3.78 → 12.50 + 3.78.
- Add from the rightmost column to the left with the same carry rules.
- Put the decimal point in the answer directly below the other decimal points.
- Example: 12.50 + 3.78 = **16.28**.

Greatest Common Divisor

✓ **Meanings:**

- **Divisor (factor) of n:** a natural number that divides n with no remainder. Every divisor of n lies between 1 and n.
- **Common divisor of m and n:** a number that divides both m and n.
- **Greatest common divisor, gcd(m, n):** the largest common divisor of m and n. It is also called the

Conceptual Learning

highest common factor (hcf).

✓ **Breaking the problem into smaller parts:**

1) Find all the divisors of each number and write them down.
2) Compare the two collections and find the largest number present in both.

✓ **List:**

- **List:** a sequence of values written in order inside square brackets, e.g. [1, 4, 9, 16]. An empty list is written [ ].
- A new value is added at the **end** (rightmost place) of the list.
- Names such as list-of-divisors and j are used for the values we keep track of. This keeps the steps short and clear.
- **Executing** (carrying out step by step) **an algorithm:** running through its steps one by one.

✓ **Algorithm to find the divisors of n:**

1) Start with an empty list-of-divisors.
2) For each number j in 1, 2, 3, …, n: if j divides n, add j to the end of list-of-divisors.

- **Basic step assumed:** we can check whether a smaller number divides a larger number.
- The list comes out in **increasing order**, because j is checked from 1 upward.

✓ **Worked examples:**

- **Example 1:** executing divisors(18).

j	Does j divide 18?	list-of-divisors
start	–	[ ]
1	Yes	[1]
2	Yes	[1, 2]
3	Yes	[1, 2, 3]
4, 5	No	[1, 2, 3]
6	Yes	[1, 2, 3, 6]
7, 8	No	[1, 2, 3, 6]
9	Yes	[1, 2, 3, 6, 9]
10 to 17	No	[1, 2, 3, 6, 9]
18	Yes	[1, 2, 3, 6, 9, 18]

- So divisors(18) = **[1, 2, 3, 6, 9, 18]**.
- **Example 2:** divisors(375) = [1, 3, 5, 15, 25, 75, 125, 375] and divisors(825) = [1, 3, 5, 11, 15, 25, 33, 55, 75, 165, 275, 825]. The largest number in both lists is **75**.

✓ **Finding the common divisors:**

- For big numbers, the lists are too long to compare by eye. For example, divisors(54000) has 80 entries and divisors(81000) has 75 entries.
- So we build a new list, **common-divisors**, starting empty. For each divisor in the first list, if it is also in the second list, add it to common-divisors.
- 1 divides every number, so common-divisors always has at least one entry, 1.

Conceptual Learning

- Both lists are in increasing order, so common-divisors is also in increasing order. The **rightmost element is the gcd**.
- For 54000 and 81000, the common divisors are [1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, …, 6750, 9000, 13500, 27000], so gcd = **27000**.

✓ **First algorithm for gcd(m, n):**

1) Let divisors-of-m be the list divisors(m).
2) Let divisors-of-n be the list divisors(n).
3) Start with an empty list common-divisors.
4) For each number x in divisors-of-m: if x is also in divisors-of-n, add x to common-divisors.
5) Report the rightmost element of common-divisors as gcd(m, n).

- **Example 3:** gcd(375, 825). Check each of 1, 3, 5, 15, 25, 75, 125, 375 in the divisors of 825.
- 1, 3, 5, 15, 25, 75 are present; 125 and 375 are not. common-divisors = [1, 3, 5, 15, 25, 75], so gcd(375, 825) = **75**.

✓ **Solved questions:**

Q8. Try to execute the algorithm to compute the divisors of 15, 135, and 775. How does the amount of work increase as the numbers grow?

➢ divisors(15): check 1 to 15 → **[1, 3, 5, 15]** (15 checks).
➢ divisors(135): check 1 to 135 → **[1, 3, 5, 9, 15, 27, 45, 135]** (135 checks).
➢ divisors(775): check 1 to 775 → **[1, 5, 25, 31, 155, 775]** (775 checks).
➢ The number of checks equals n itself.
➢ So the work grows **in proportion to the value** of the number: a number 10 times bigger needs 10 times more checks.

Data Structures and Better gcd Algorithms

✓ **Data structure:**

- While executing an algorithm, we compute in-between values such as divisors-of-m and divisors-of-n, and give them names so that we can use them later.
- These values are often more than single numbers, e.g. lists arranged in ascending (increasing, from small to large) order.
- **Data structure:** a way of organising information (such as a list) that helps an algorithm work efficiently.

✓ **Improvement 1: compute both lists together:**

- The first algorithm checks 1 to m for divisors of m, then 1 to n for divisors of n: a total of **m + n** checks.
- Instead, run j from 1 to **max(m, n)** (the larger number) once: if j divides m, add it to divisors of m; if j divides n, add it to divisors of n.
- Checks needed: **max(m, n)** instead of m + n.

✓ **Improvement 2: build the common divisors directly:**

- For each j from 1 to max(m, n): if j divides **both** m and n, add j to common-divisors.
- A common divisor can never be larger than the smaller number. So it is enough to check j from 1 to **min(m, n)**.

Conceptual Learning

✓ **Improvement 3: no lists at all:**

- Once a larger common divisor is found, the smaller ones found before it are of no use.
- So keep only one value, **most-recent-common-divisor**, starting at 1 (1 always divides both numbers).
- For each k from 2 to min(m, n): if k divides both m and n, set most-recent-common-divisor = k.
- At the end of the scan, most-recent-common-divisor is the **gcd**.
- **Example:** m = 6, n = 12, min(6, 12) = 6.

k	Divides 6 and 12?	most-recent-common-divisor
start	–	1
2	Yes	2
3	Yes	3
4	No (does not divide 6)	3
5	No (does not divide 6)	3
6	Yes	6

- Scan complete: gcd(6, 12) = **6**.

✓ **Analysing these algorithms:**

- **Correctness:** the first algorithm follows directly from what gcd means, so its answer is surely the gcd. Each improvement saves work but computes the same thing, so all versions give the correct gcd.
- **Efficiency:** even the best version checks every number from 1 to min(m, n).
- If the smaller number grows from 3 digits to 4 digits, about **10 times** as many checks are needed; from 3 to 5 digits, about **100 times**.
- This is like adding by counting dots. The aim is an algorithm whose work grows with the **number of digits**, as in column addition.

✓ **Solved questions:**

Q9. Suppose List 1 and List 2 are two lists of numbers in increasing order. (i) Write an algorithm to find elements in List 1 that are not present in List 2. (ii) Write an algorithm to find elements in List 2 that are not present in List 1.

➢ (i) Start with an empty list only-in-1.
➢ For each number x in List 1: if x does **not** appear in List 2, add x to only-in-1.
➢ Report only-in-1.
➢ (It stays in increasing order, as List 1 is.)
➢ (ii) Start with an empty list only-in-2.
➢ For each number y in List 2: if y does not appear in List 1, add y to only-in-2.
➢ Report only-in-2.
➢ Example: List 1 = [1, 3, 5, 15], List 2 = [1, 2, 3, 6].
➢ (i) gives [5, 15]; (ii) gives [2, 6].

Q10. Describe an algorithm to compute the least common multiple (lcm) of two numbers.

➢ **Meaning:** lcm(m, n) is the smallest number that is a multiple of both m and n.
➢ It lies between max(m, n) and m × n.
➢ Let big = max(m, n) and small = min(m, n).

Conceptual Learning

- For each k = 1, 2, 3, …: compute big × k.
- If small divides big × k, report big × k as the lcm and stop.
- Example: lcm(6, 8): 8 × 1 = 8 (6 does not divide), 8 × 2 = 16 (no), 8 × 3 = 24 (yes). lcm = **24**.
- Shorter method: lcm(m, n) = (m × n) ÷ gcd(m, n)
- For 6 and 8: 48 ÷ 2 = 24.

Q11. Divisors occur in pairs. For instance, the divisors of 18 are (1, 18), (2, 9) and (3, 6). (i) If we write out divisors in pairs, how many numbers do we have to examine between 1 and n to find all the divisors of n? (ii) If we list out the divisors in pairs, will our gcd algorithm still work in the manner we have described?

- (i) If j divides n, then n ÷ j also divides n.
- In each pair, the smaller number is at most √n (because √n × √n = n).
- So we only need to check j from 1 to **√n**, and add both j and n ÷ j.
- For 18: check 1 to 4 only (√18 ≈ 4.2).
- (ii) **Not as described.** Listed in pairs, the divisors are not in increasing order: [1, 18, 2, 9, 3, 6].
- So common-divisors is also not in increasing order, and its rightmost element need not be the largest.
- Fix: report the **largest** element of common-divisors (or sort the lists first) instead of the rightmost element.

Q12. How would our original algorithm change if we computed the divisors of n by examining the numbers from 1 to n in reverse order, from n down to 1?

- Each list of divisors would be in **decreasing** order, e.g. divisors(18) = [18, 9, 6, 3, 2, 1].
- The list common-divisors would also be in decreasing order.
- So the gcd would be the **leftmost (first)** element of common-divisors, not the rightmost.

Q13. What about the last algorithm described above (without lists)? What happens when we look at common divisors starting from min(m, n) and work backwards to 1?

- The first number k that divides both m and n is the largest common divisor, since every number above it has already been checked.
- So we can **stop as soon as the first common divisor is found**; that k is the gcd.
- Example: gcd(6, 12): k = 6 divides both, so the answer is 6 after just one check.
- This often saves a lot of work, though in the worst case (gcd = 1) all numbers down to 1 are still checked.

Euclid's Subtraction Algorithm

✓ **History:**

- Euclid wrote this method down in his work *Elements*. Other Greek mathematicians probably knew it before him, but he was the first to write it down.

✓ **Key fact: gcd(m, n) = gcd(n, m − n):**

- Let m ≥ n.
- If d divides m and n, then m = ad and n = bd for natural numbers a, b. So m − n = (a − b)d, and d divides m − n.
- If d divides n and m − n, then n = xd and m − n = yd. So m = n + (m − n) = (x + y)d, and d divides m.
- So m, n and n, m − n have **exactly the same common divisors**, and hence the same gcd.

Conceptual Learning

✓ **Tiling picture:**

- If blocks of length d exactly cover (tile) both m and n, then the leftover part m − n is also exactly covered by blocks of length d. Euclid used a similar picture.

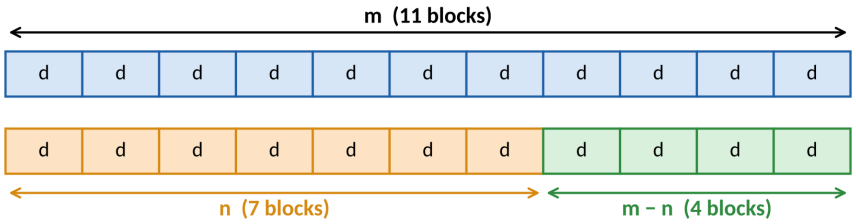


Fig. 3: If d tiles m and n, then d also tiles m − n

✓ **Reducing a problem to a simpler one:**

- The problem gcd(m, n) is replaced by gcd(n, m − n), which has smaller numbers.
- The same step is repeated on the new pair, again and again.
- **Stopping point:** gcd(m, 0) = m. (For example, gcd(23, 23) reduces to gcd(23, 0) = 23.)

✓ **Euclid's algorithm for gcd(m, n):**

1) If m < n, swap the numbers and compute gcd(n, m).
2) If n = 0, report the answer as m.
3) Otherwise, reduce the problem to gcd(n, m − n).

✓ **Worked examples:**

- **Example 1:** gcd(375, 825).

1) 375 < 825, so swap: gcd(825, 375).
2) 825 − 375 = 450: gcd(375, 450); swap: gcd(450, 375).
3) 450 − 375 = 75: gcd(375, 75).
4) 375 − 75 = 300: gcd(75, 300); swap: gcd(300, 75).
5) 300 − 75 = 225: gcd(75, 225); swap: gcd(225, 75).
6) 225 − 75 = 150: gcd(75, 150); swap: gcd(150, 75).
7) 150 − 75 = 75: gcd(75, 75). Then 75 − 75 = 0: gcd(75, 0).
8) n = 0, so the answer is **75**. (7 reductions in all.)

- **Example 2:** gcd(99, 2) → gcd(97, 2) → gcd(95, 2) → … → gcd(3, 2) → gcd(2, 1) → gcd(1, 1) → gcd(1, 0) = **1**.
- For an odd number 2k + 1, gcd(2k + 1, 2) needs about k reductions. So the work grows with the **value** of the number, not with its number of digits.

✓ **Solved questions:**

Q14. Find gcd(48, 18) using Euclid's subtraction algorithm.

➢ gcd(48, 18) → gcd(18, 30) → swap → gcd(30, 18).
➢ gcd(30, 18) → gcd(18, 12).
➢ gcd(18, 12) → gcd(12, 6).
➢ gcd(12, 6) → gcd(6, 6) → gcd(6, 0).
➢ n = 0, so gcd(48, 18) = **6**.

Conceptual Learning

Āryabhaṭa's Division Algorithm

✓ **History:**

- Āryabhaṭa's *Āryabhaṭīya* (499 CE) and other Indian works use the "long division" method to find the gcd with Indian place-value numerals.
- It uses repeated division (taking the remainder) in place of repeated subtraction.

✓ **m mod n:**

- **m mod n:** the remainder when m is divided by n. Examples: 17 mod 5 = 2; 33 mod 7 = 5; 99 mod 2 = 1.
- Subtracting n from m again and again until it can no longer be subtracted leaves exactly m mod n. So one division does the work of many subtractions.

✓ **Key fact: gcd(m, n) = gcd(n, m mod n):**

- Let m = qn + r, where r = m mod n.
- If d divides m and n, it divides m − qn = r.
- If d divides n and r, it divides qn + r = m.
- So d tiles m and n exactly when it tiles n and m mod n. The common divisors, and hence the gcd, are the same.

✓ **Division algorithm for gcd(m, n):**

1) If m < n, swap the numbers and compute gcd(n, m).
2) If n = 0, report the answer as m.
3) Otherwise, reduce the problem to gcd(n, m mod n).

- Only Step 3 has changed; the stopping rule is the same.

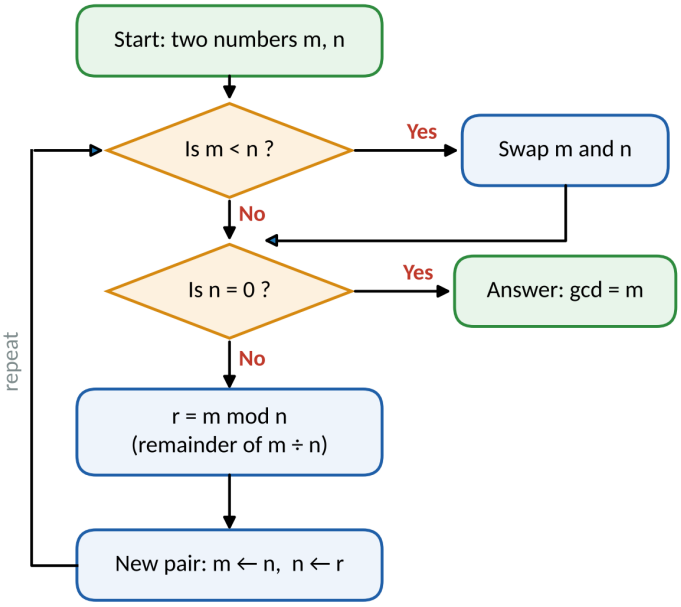


Fig. 4: Flow chart of the division algorithm for the gcd

✓ **Long division layout:**

- Divide the larger number by the smaller. The remainder becomes the new divisor, and the old divisor becomes the new dividend.
- Repeat until the remainder is 0. The last divisor is the gcd.

gcd(99, 2) = 1

```
2)99(49
  98
  --
 1)2(2
   2
   -
   0
```

gcd(825, 375) = 75

```
375)825(2
  750
  ---
 75)375(5
  375
  ---
    0
```

gcd(60, 16) = 4

```
16)60(3
  48
  --
 12)16(1
   12
   --
  4)12(3
    12
    --
     0
```

Each remainder (red) becomes the next divisor. The divisor that leaves remainder 0 is the gcd.

Fig. 5: Long division method for gcd(99, 2), gcd(825, 375) and gcd(60, 16)

✓ **Speed of the two methods:**

- gcd(99, 2) needs only **2** reductions: gcd(99, 2) → gcd(2, 1) → gcd(1, 0) = 1. The subtraction method needs about 50.
- For any gcd(2k + 1, 2): (2k + 1) mod 2 = 1, so gcd(2k + 1, 2) → gcd(2, 1) → gcd(1, 0). Always 2 reductions.
- In general, the number of reductions in Āryabhaṭa's division algorithm grows with the **number of digits** of m and n, not their value. (The proof comes in higher grades.)

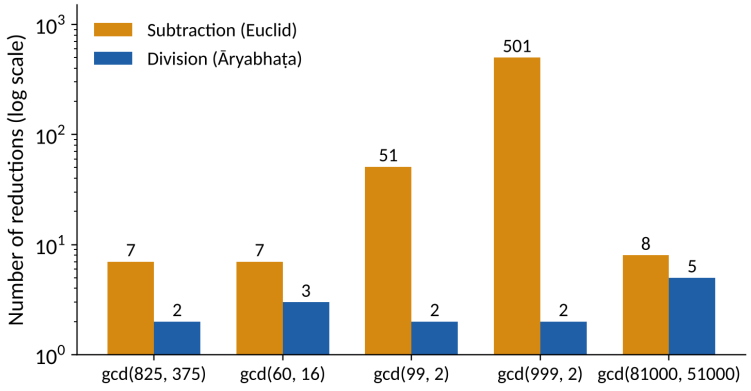


Fig. 6: Number of reductions: subtraction method versus division method

✓ **Worked examples:**

- **Example 1:** gcd(60, 16): 60 = 3 × 16 + 12 → gcd(16, 12); 16 = 1 × 12 + 4 → gcd(12, 4); 12 = 3 × 4 + 0 → gcd(4, 0) = **4**.
- **Example 2:** gcd(825, 375): 825 = 2 × 375 + 75 → gcd(375, 75); 375 = 5 × 75 + 0 → gcd(75, 0) = **75**. Only 2 reductions, against 7 by subtraction.

✓ **Solved questions:**

Q15. Compute the following using the improved version of Euclid's algorithm. (i) gcd(375, 825) (ii) gcd(51000, 81000) (iii) gcd(1789287, 237656) (iv) gcd(2587392, 157656)

➢ (i) Swap: gcd(825, 375). 825 = 2 × 375 + 75 → gcd(375, 75). 375 = 5 × 75 + 0 → gcd(75, 0)

- **gcd = 75**.
- (ii) Swap: gcd(81000, 51000). 81000 = 1 × 51000 + 30000 → gcd(51000, 30000). 51000 = 1 × 30000 + 21000 → gcd(30000, 21000). 30000 = 1 × 21000 + 9000 → gcd(21000, 9000). 21000 = 2 × 9000 + 3000 → gcd(9000, 3000). 9000 = 3 × 3000 + 0.
- **gcd = 3000**.

m	n	Quotient	m mod n
1789287	237656	7	125695
237656	125695	1	111961
125695	111961	1	13734
111961	13734	8	2089
13734	2089	6	1200
2089	1200	1	889
1200	889	1	311
889	311	2	267
311	267	1	44
267	44	6	3
44	3	14	2
3	2	1	1
2	1	2	0

- (iii) The table above shows each division.
- The last non-zero divisor is 1, so **gcd(1789287, 237656) = 1** (the numbers are co-prime). 13 reductions were needed.

m	n	Quotient	m mod n
2587392	157656	16	64896
157656	64896	2	27864
64896	27864	2	9168
27864	9168	3	360
9168	360	25	168
360	168	2	24
168	24	7	0

- (iv) From the table, the last divisor before remainder 0 is 24.
- So **gcd(2587392, 157656) = 24**, after 7 reductions.

Q16. Assume m ≥ n. Verify that d divides m and n if and only if d divides both n and m mod n.

- Write m = qn + r, where q is the quotient and r = m mod n (0 ≤ r < n).
- **If d divides m and n:** m = ad and n = bd.
- Then r = m − qn = ad − qbd = (a − qb)d.
- So d divides r = m mod n, and d already divides n.
- **If d divides n and r:** n = xd and r = yd.

Conceptual Learning

- Then m = qn + r = qxd + yd = (qx + y)d.
- So d divides m, and d already divides n.
- Both directions hold, so d divides m and n **if and only if** d divides n and m mod n.
- Check: m = 60, n = 16, m mod n = 12.
- Common divisors of 60, 16: 1, 2, 4.
- Common divisors of 16, 12: 1, 2, 4.
- Same ✓.

Q17. With the division algorithm, gcd(99, 2) needs only two reduction steps. Check that this would happen for any similar problem of the form gcd(2k + 1, 2).

- 2k + 1 is odd, so (2k + 1) mod 2 = 1.
- First reduction: gcd(2k + 1, 2) → gcd(2, 1).
- 2 mod 1 = 0.
- Second reduction: gcd(2, 1) → gcd(1, 0).
- n = 0, so the answer is 1.
- Always **2 reductions**, however large k is.

More Algorithms on Divisors and Primes

✓ **Prime number:**

- **Prime number:** a number with exactly two distinct divisors, 1 and itself. 1 is not prime (it has only one divisor).

✓ **Solved questions:**

Q18. Write an algorithm prime(n) to check if n is prime. (Hint: A prime number p has exactly two distinct factors, 1 and p. Can you make use of divisors(n) to write out prime(n)?)

- Compute the list divisors(n).
- Count the number of elements in this list.
- If the count is exactly 2, report "n is prime"; otherwise report "n is not prime".
- Examples: divisors(13) = [1, 13], count 2 → **prime**. divisors(15) = [1, 3, 5, 15], count 4 → **not prime**. divisors(1) = [1], count 1 → not prime.

Q19. Write an algorithm primedivisors(n) to compute the list of divisors of n that are prime numbers. (Hint: Compute divisors(n) and then filter out the primes in this list.)

- Compute the list divisors(n).
- Start with an empty list prime-divisors.
- For each number x in divisors(n): if prime(x) says x is prime, add x to prime-divisors.
- Report prime-divisors.
- Example: divisors(60) = [1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60].
- Keeping only primes gives primedivisors(60) = **[2, 3, 5]**.

Q20. We can also find the gcd of two numbers by computing the prime factorisation of both the numbers. Try to write an algorithm to compute the prime factorisation of a number. (i) The prime factorisation of 180 is $2^2 \times 3^2 \times 5^1$. How would you represent this? (ii) How would you compare the prime factorisations of two numbers?

Conceptual Learning

- **Algorithm factorise(n):** start with an empty list factors and set p = 2.
- While n > 1: if p divides n, add p to factors and replace n by n ÷ p (keep the same p); otherwise increase p by 1.
- Report factors.
- Example: 180 → 2, 90 → 2, 45 → 3, 15 → 3, 5 → 5, 1. factors = [2, 2, 3, 3, 5].
- (i) Represent it as a list of (prime, power) pairs: **[(2, 2), (3, 2), (5, 1)]**.
- (ii) Write both numbers as such lists.
- For each prime present in **both** lists, take the **smaller** power.
- The product of these prime powers is the gcd.
- Example: 180 = [(2, 2), (3, 2), (5, 1)] and 84 = [(2, 2), (3, 1), (7, 1)].
- Common primes 2 and 3 with smaller powers 2 and 1: gcd = $2^2 \times 3$ = **12**.
- (Taking every prime present in **either** list with the **larger** power gives the lcm.)

Conceptual Learning